SANCTIFY

WORDPRESS SECURITY • INCIDENT RESPONSE

Malware Infection & Remediation Report

Casino Pride — https://www.cpofficial.in

Threat:	“Smooth Backup Ink” (SCV) self-healing PHP malware
Severity:	Critical
Status:	RESOLVED — site clean, active protection deployed
Report date:	5 September 2026
Prepared by:	Sanctify • www.sanctify.in

Executive Summary

On 5 September 2026, Sanctify performed a security assessment of the WordPress website **Casino Pride (https://www.cpofficial.in)** and identified an active, **self-healing malware infection** belonging to the "Smooth Backup Ink" (internal marker SCV) family. The malware executed on every page request, disguised itself as a legitimate plugin, and could rebuild itself after any file was deleted — which is why earlier cleanup attempts had not held.

Sanctify fully removed the infection, eliminated every persistence and re-entry mechanism (including a database-level cron backdoor and a rogue administrator account), verified that the site no longer regenerates the malware, and deployed a purpose-built defensive plugin — **Sanctify Falcon** — to prevent recurrence. The website is now clean and serving normal content.

Area	Outcome
Infection	Fully removed and verified
Persistence (cron / users / plugins)	Eliminated at the source
Re-infection test	Passed — no regeneration after forcing wp-cron
Ongoing protection	Sanctify Falcon plugin installed & active
Website availability	Online throughout; content & layout intact

1. The Infection

Nature of the threat

The site was infected with a sophisticated PHP malware that masqueraded as a fake plugin named **"Smooth Backup Ink"**. Rather than a single malicious file, it was a **multi-layered, self-repairing system** engineered to survive cleanup. Its defining trait was persistence: even if the visible payload was deleted, hidden components and a scheduled task quietly recreated it.

How it worked (attack chain)

- **Execution trigger:** hidden `.user.ini` and `.htaccess` directives forced a malicious loader to run before *every* PHP page load, invisibly.
- **Loader & dropper:** the loader called a hidden, obfuscated "dropper" that checked whether the payload existed and **recreated it** from any surviving copy.
- **Payloads:** multiple large encoded files (a fake mu-plugin auto-loaded by WordPress, a fake database dropin, and hidden state files) carried the actual malicious logic.
- **Database backdoor:** scheduled WordPress cron tasks re-dropped the files on a timer, triggered by ordinary visitor traffic — the true reason it kept returning.
- **Access backdoor:** a rogue administrator account allowed the attacker to simply log back in.

Why previous cleanups failed

Deleting the malware files alone was never enough: the scheduled database task and the backdoor admin account remained, so the infection regenerated automatically. A permanent fix required removing the **root cause** — the persistence in the database and the unauthorized access — not just the symptoms on disk.

2. Detailed Findings

Component	Description	Severity
Malicious auto-run triggers	`.user.ini` & `.htaccess` forcing a loader on every request	**Critical**
Loader + hidden dropper	Regenerated the payload from backup copies (self-heal)	**Critical**
Fake mu-plugin & dropin	Large encoded payloads auto-loaded by WordPress	**Critical**
Malicious cron tasks	Scheduled DB events re-dropping files on a timer	**Critical**
Rogue administrator	Auto-generated `backup_*` admin account (attacker access)	**Critical**
Planted active plugin	Attacker-installed “Easypost” plugin	**High**
Re-infection seed	Zipped payload staged inside an old theme folder	**High**
Fake “loader” plugins	Empty attacker-scaffolded plugin directories	**Medium**

Confirmed clean

The active theme and its `functions.php` (integrity-verified), `wp-config.php`, WordPress core, and legitimate plugins/uploads were examined and found **free of injected code**. The website’s public HTML and JavaScript served to visitors contained no malicious scripts or redirects.

3. How Sanctify Resolved It

Remediation followed a disciplined order: **neutralise the triggers first** (to stop the self-heal), then remove payloads, then eliminate the database and access backdoors, and finally verify and harden. Every file changed was backed up beforehand, and the site remained online throughout.

#	Action taken	Result
1	Neutralised malicious `.user.ini` and cleaned injected `.htaccess` (kept all legitimate rules)	Execution trigger disabled
2	Deleted loader, dropper, and all payload files; removed hidden malware directory and fake plugin	Payloads removed
3	Cleared 5 malicious scheduled tasks from the database	Self-heal engine stopped
4	Removed the rogue administrator account (content reassigned safely)	Attacker access revoked
5	Removed the planted plugin, the zipped re-infection seed, and empty fake plugins	Re-entry seeds removed
6	Hardened `wp-content` to block direct PHP execution	Attack class blocked
7	Built & deployed the Sanctify Falcon protection plugin	Continuous defense active

Verification

To prove the fix is permanent, Sanctify explicitly triggered the WordPress scheduler (`wp-cron.php`) — the exact mechanism the malware used to regenerate — and waited. **Nothing reappeared.** The malicious files, directories, triggers, and cron tasks all stayed gone, and the website continued to return normal pages (HTTP 200) with correct content.

4. Ongoing Protection — Sanctify Falcon

Sanctify developed and installed a custom WordPress security plugin, **Sanctify Falcon** (www.sanctify.in), built specifically to defend against this malware family and similar self-healing infections. It runs automatically on every request and on an hourly schedule.

Active defenses

- **File defense** — automatically removes known dropper/loader/payload files if they ever reappear.
- **Trigger guard** — sanitises malicious auto-run directives re-injected into `.user.ini` or `.htaccess`.
- **Cron cleaner** — removes malicious and rogue scheduled tasks while protecting all legitimate ones.
- **Backdoor-admin detection** — detects auto-generated `backup_*` admin accounts, demotes them, and flags them for review.
- **Uploads quarantine** — isolates any dangerous PHP dropped into the media folder.
- **Dashboard & logging** — a “Sanctify Falcon” admin screen with an on-demand scan and a full activity log.

The plugin is intentionally conservative: it never deletes user accounts (it demotes and flags), and only acts on files/tasks that match strict malware signatures, with an allow-list protecting legitimate plugin files.

5. Recommendations

The infection is resolved and active protection is in place. To fully close the loop, Sanctify recommends the following owner actions:

- **Rotate all credentials** — WordPress administrators, hosting/FTP, and database password (with fresh security keys). Revoke any temporary access provided for this engagement.
- **Update or replace higher-risk plugins** — file-manager, slider, and theme-addon plugins are common entry points; ensure they are on the latest versions or removed if unused.
- **Reinstall WordPress core** of the same version to guarantee core file integrity.
- **Keep Sanctify Falcon active** and review its dashboard/log periodically.
- **Remove any nulled/pirated plugins or themes**, the most frequent source of such infections.
- **Request reputation review** with Google Search Console and Malwarebytes so any residual “flagged” status from the infection period is cleared.

Appendix — Indicators of Compromise (IOCs)

For future monitoring, the following indicators identify this malware family:

Type	Indicators
Files / dirs	`64f6b5eb.php, .64f6b5eb.php, ed0c7e2b.php, db.php (SC_DB), 865fa429.zip, bce0f3b9.php, .sc_<hex>/, mu-plugins/smooth-backup-ink.php`
Cron hooks	`sc_cron_fetch, my_monitoring_cron`, random 16+ character hex hook names
Users	`backup_<hex>` administrator accounts with auto-generated emails
Content markers	`SCV:4.3.24, SC_DB_BEGIN, Smooth Backup Ink, auto_prepend_file`

— End of report —
Prepared by Sanctify • www.sanctify.in • 5 September 2026